

AIML - PGCP: Capstone Project

The team-based Capstone Project completion is the objective of the PG Certification in AIML program. It will provide an opportunity for the participants to implement an end-to-end AIML project. The expectation from this Capstone Project is that the participants should be able to apply the learnings from the Advanced Certification program and demonstrate a full-fledged deployed AIML solution for the selected Problem Statement/Projects as given below.

PROJECT 1

Title: Multimodal Media Retrieval and Captioning System

Objective: Build a model capable of retrieving images and text based on cross-modal queries and generating descriptive captions for images lacking textual information.

Dataset Link: [COCO Dataset](#), [Conceptual Captions Dataset](#)

Dataset Description:

A collection of diverse images with corresponding textual captions. The COCO dataset consists of 330k images, each annotated with five captions, capturing various objects and scenes. The Conceptual Captions dataset has millions of images with high-level, real-world context descriptions, ideal for training robust caption generation and retrieval tasks.

Project Overview:

Retrieving relevant visual and textual information based on cross-modal queries is a crucial challenge in AI, enhancing applications like content recommendation, digital archiving, and accessibility tools. This project involves creating a unified multimodal neural network that enables image-to-text and text-to-image retrieval, alongside generating accurate captions for images. The project will leverage both Computer Vision and Natural Language Processing techniques, embedding visual and textual data in a shared space for seamless multimodal interaction. The system's practical use cases include enhancing digital media archives, improving accessibility for visually impaired users, and providing content recommendations. The model will be evaluated on retrieval and captioning tasks, and the deployment will support real-time content curation applications.

Tools: Natural Language Toolkit, TensorFlow, PyTorch, Keras

Deployments: FastAPI, Cloud Application Platform | Heroku, Streamlit, Cloud Computing, Hosting Services, and APIs | Google Cloud

Final Submissions:

- GitHub Repository of the project
- Project Technical Report
- Project Presentation with desired outcomes
- Summary of 3 research papers

PROJECT 2

Project Title: Real World RAG System

Objective:

Develop an efficient Retrieval-Augmented Generation (RAG) pipeline capable of delivering accurate answers to the user queries.

Dataset: <https://huggingface.co/datasets/rungalileo/ragbench>

Dataset Description: RAG Bench is a large-scale benchmark dataset specifically designed for training and evaluating RAG systems. It contains 100k examples spanning five industry-specific domains:

- Biomedical Research
- General Knowledge
- Legal
- Customer Support
- Finance

The dataset supports various RAG task types and includes evaluation metrics such as context relevance, context utilization, answer faithfulness and answer completeness.

The dataset samples are sourced from real-world industry corpora, such as user manuals, making the dataset highly relevant for practical applications. This comprehensive dataset enables a detailed assessment of RAG system performance, ensuring robust and reliable evaluation.

Project Overview: Generative large language models often face challenges such as producing outdated information or fabricating facts. Retrieval-Augmented Generation (RAG) techniques address these limitations by integrating pretraining and retrieval-based approaches. This combination creates a robust framework for improving model accuracy and reliability.

RAG offers the added benefit of enabling rapid deployment of domain-specific applications without requiring updates to the model parameters. As long as relevant documents are available for a query, the system can adapt to organizational or domain-specific needs efficiently.

A typical RAG workflow involves Query Classification, Retrieval, Reranking, Repacking and Summarization. Implementing RAG requires careful consideration of various factors, such as properly splitting documents into manageable chunks, selecting suitable embeddings to semantically represent these chunks, and choosing vector databases for efficient storage and retrieval of feature representations.

By addressing these elements, RAG systems can deliver accurate, domain-specific, and context-aware responses, making them a powerful tool for real-world applications.

References:

- 1) [Searching for Best Practices in Retrieval-Augmented Generation](#),
- 2) [RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems](#)

Tools: Transformers, Hugging Face, PyTorch, Langchain, LlamaIndex

Deployments: FastAPI, Cloud Application Platform | Heroku, Streamlit, Cloud Computing, Hosting Services, and APIs | Google Cloud

Final Submissions:

- Project technical report & presentation with desired outcomes
- An overview of the modeling techniques used for the problem
- GitHub Repository of the project
- Summary of 3 research papers

PROJECT 3

Title: Intelligent Retail Product Detection and Inventory Management System using Computer Vision and LLM-Based Business Intelligence

Objective: Develop an end-to-end AI-powered retail inventory management system capable of accurately detecting and recognizing products in dense retail shelf images, maintaining real-time inventory levels, and providing natural language interfaces for business intelligence queries.

Dataset Link:

SKU-110K Dataset: <https://www.kaggle.com/datasets/thedatasith/sku110k-annotations>
https://github.com/eg4000/SKU110K_CVPR19

RPC (Retail Product Checkout) Dataset:

<https://www.kaggle.com/datasets/diyer22/retail-product-checkout-dataset>
<https://rpc-dataset.github.io>

Dataset description:

SKU-110K is a large-scale dataset specifically designed for object detection in densely packed retail environments. It contains over 110,000 unique store keeping unit (SKU) images captured from retail store shelves worldwide. Key characteristics include:

- **Dense Object Scenarios:** Images contain extremely crowded scenes with dozens to hundreds of products per image
- **Visual Similarity Challenges:** Products often have similar packaging, colors, and shapes
- **Real-world Conditions:** Includes variations in lighting, angles, occlusions, and shelf arrangements
- **Bounding Box Annotations:** Each product instance is annotated with precise bounding boxes

The dataset addresses critical challenges in retail environments such as high object density, partial occlusions, scale variance, and visual similarity between different SKUs. These characteristics make it ideal for training robust detection models that can handle real-world retail scenarios.

RPC (Retail Product Checkout) is a large-scale dataset for fine-grained product recognition at checkout counters. It contains:

- **Product Categories:** 200 different product categories commonly found in retail stores
- **Multiple Instances:** Images contain varying numbers of products (1 to 20+) per image

- **Checkout Scenarios:** Simulates real checkout counter conditions with products placed in various arrangements
- **Rich Annotations:** Includes product category labels, bounding boxes, and instance counts

The RPC dataset complements SKU-110K by providing fine-grained classification challenges and supporting downstream tasks like automatic billing and inventory counting.

Additional datasets such as ImageNet and COCO are used for transfer learning and pre-training detection backbones, providing general object recognition capabilities that transfer well to retail-specific applications.

Project Overview:

Traditional retail inventory management faces several critical challenges: manual product counting is time-consuming and error-prone, stock-taking requires significant labor hours, real-time inventory tracking is difficult to maintain, and business intelligence typically requires technical SQL knowledge.

Computer vision-based approaches offer a promising solution by automating product detection and recognition from shelf images. However, accurate product recognition in dense retail images presents unique challenges including high object density and clutter, visual similarity between different SKUs, partial occlusions from overlapping products, and scale variance from small candies to large boxes.

This project addresses these challenges through a multi-stage detection and classification pipeline. The workflow begins with universal object detection to identify all products in an image, followed by hierarchical classification to narrow down product categories, and finally fine-grained classification to identify specific products. This cascaded approach improves accuracy while maintaining computational efficiency.

For products not in the existing database, the system leverages vision-language models to automatically extract product information from packaging images. This auto-labeling capability enables the system to continuously learn and adapt to new products without manual intervention.

The system also implements a natural language query interface powered by large language models. Users can ask business questions in plain English (e.g., "Show products running low on stock this week") without requiring SQL knowledge. The LLM converts these queries to appropriate database operations and returns formatted results.

To ensure enterprise readiness, the system includes multi-layer security guardrails to prevent prompt injection attacks, input validation, and output sanitization. Additional features include automated bill generation, low-stock alerts, and comprehensive analytics dashboards.

By combining object detection, representation learning, vision-language models, and conversational AI, this project demonstrates how modern AI techniques can be effectively

integrated to solve real-world retail problems while maintaining high accuracy, low latency, and strong security.

References:

1. Goldman, E., et al. (2019). Precise Detection in Densely Packed Scenes. CVPR
2. Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML
3. Liu, H., et al. (2023). Visual Instruction Tuning. NeurIPS

Tools:

YOLO (Ultralytics), PyTorch, TensorFlow, OpenCV, CLIP, LLaVA, Ollama, LangChain, LangGraph, Scikit-learn, FastAPI, Streamlit, SQLite

Deployments:

FastAPI, Docker, Cloud Application Platform | Heroku, Streamlit, Cloud Computing, Hosting Services, and APIs | Google Cloud

Final Submissions:

- GitHub Repository of the project
- Project Technical Report
- Project Presentation with desired outcomes

PROJECT 4

Title: Automatic Speech Recognition(ASR)

Objective: Build an ASR model for converting speech to text.

Dataset Link : [LibriSpeech](#)

Dataset description: LibriSpeech is a corpus of reading English speech, suitable for training and evaluating speech recognition systems, published in 2015 by Johns Hopkins University. It is derived from audiobooks that are part of the LibriVox project and contains 1000 hours of speech sampled at 16 kHz of 2000 speakers. The LibriVox project¹, a volunteer effort, is responsible for the creation of approximately 8000 public domain audiobooks, the majority of which are in English. Most of the recordings are based on texts from Project Gutenberg², also in the public domain. The data is already divided into train/dev/test sets. The total size of the data is 60 GB and subsets are available of different sizes.

Initially, we recommend working only with 'dev-clean' and 'test-clean' datasets for building the model. We can use any one or a combination of both data sets as a training set. A subset of either 'dev-clean' or 'test-clean' can be used for testing purposes. Once modeling is done with these smaller data sets, start modeling using 'train-clean'/'train-other' data sets of larger sizes as a training set. Now, 'dev-clean', 'test-clean', and 'test-other' datasets are used for validation/testing purposes only.

Project Overview: Automatic speech recognition is the application of Machine learning or AI where human speech is processed and converted into readable text. We can find numerous applications, such as Instagram for real-time captions, Spotify for podcast transcriptions, YouTube video transcription, Zoom meeting transcriptions, etc. The field has grown exponentially over the last few years. An explosion of applications is taking advantage of ASR technology in their products to make audio and video data more accessible.

There are different approaches to Automatic Speech Recognition, viz., traditional HMM (Hidden Markov Models), GMM (Gaussian Mixture Models), end-to-end deep learning, and Self-supervised models. In this project, we aim to build and deploy a model that can generate the written text from the speech with decent accuracy. The main aim is to build and deploy a self-supervised model, such as wav2vec, a state-of-the-art ASR model that is built on a Transformer architecture and uses the concept of contrastive learning.

Tools: PyTorch, Audio Processing tool/library, fairseq.

Operating system: Linux - Ubuntu

Deployments: FastAPI, Cloud Application Platform | Heroku, Streamlit, Cloud Computing, Hosting Services, and APIs | Google Cloud

Reference: [Papers using libriSpeech](#)

Final Submissions:

- GitHub Repository of the project
- Project Technical Report
- Project Presentation with desired outcomes
- Summary of 3 research papers

PROJECT 5

Title: Text-To-Speech (TTS)

Objective: Build a TTS model that converts input text into natural-sounding speech.

Recommended Dataset

Use the LJSpeech dataset (or HiFi-TTS, VCTK, depending on your focus).

- LJSpeech: 13,100 short audio clips of a single speaker reading passages from 7 non-fiction books.
- VCTK: Contains speech data from 100 English speakers with various accents.
- HiFi-TTS / LibriTTS: Extensions with more speakers and higher fidelity.

Dataset link: <https://keithito.com/LJ-Speech-Dataset/>

Dataset Description

- .wav files: raw speech audio
- .txt or .csv/.json files: corresponding transcripts
- Sampling rate: 22.05 kHz (for LJSpeech) / 24 kHz / 16 kHz depending on the dataset.
- For initial experimentation, use a subset of data (e.g., 1000 clips) and scale up later.

Project Overview

Text-to-Speech is the process of converting written text into spoken voice. It has wide applications in: Virtual assistants (Siri, Alexa), Accessibility (screen readers), Interactive voice response (IVR), Audiobook generation, Language learning tools

These models typically transform text into intermediate acoustic features like spectrograms, which are then converted into waveforms using vocoders like HiFi-GAN or WaveGlow. TTS enhances accessibility and user experience by enabling machines to communicate effectively through speech.

TTS has evolved from traditional signal processing techniques to powerful end-to-end deep learning and self-supervised approaches.

You'll explore modern neural TTS architectures such as:

1. **Tacotron 2** (Text → Mel spectrogram → Waveform)
2. **Fastpitch** (non-autoregressive models for faster synthesis)

Tools & Frameworks

1. PyTorch
2. TTS by Coqui

3. TensorBoard / WandB for visualization

Operating System

- Linux - Ubuntu (preferred for GPU support and compatibility)

Deployment

You can deploy the TTS model using:

- FastAPI or Flask to serve the model as an API
- Streamlit for a simple front-end UI
- Save output audio as .wav/.mp3 for users to play/download

Final Submissions:

- GitHub Repository of the project
- Project Technical Report
- Project Presentation with desired outcomes
- Summary of 3 research papers

References:

1. <https://arxiv.org/abs/1712.05884>
2. <https://arxiv.org/abs/2006.06873>

PROJECT 6

Title: CodeGen

Objective: Language models are increasingly used beyond natural language to understand and synthesize programming languages. This has multiple very important applications, including automating code generation and documentation, understanding and transforming large legacy codebases, making data querying and visualization interactive, etc. The goal is to apply existing models to solve various software-related tasks and further fine-tune an existing model to expand it to a new language that is not already supported by the model.

Tasks:

1. Evaluate the program synthesis and documentation quality of a given small code LM.
 1. Generate a program for a given problem description.
 2. Generate documentation for a given program.
 3. Generate commit messages from file diffs.
2. Evaluate interactive querying of databases using small code LM.
 1. Generate the SQL query from text and evaluate it.
3. Compare the performance of small code LM against that of an LLM
 1. Measure improvements from using a RAG-based system.
 2. Index some programs of the same language and use it for RAG to evaluate gains from it. Use embeddings from a small code LM for semantic search. Also consider indexing abstract syntax trees and using it for retrieval.
 3. Draw the top-K (test with various K) similar docs from the index for each test query to pass to LLM and evaluate gains from this approach.

Resources

1. Model candidates: Start with the codegen-350M-multi or similar for a small code LM. Use embeddings from the same small code LM for indexing and querying. Use any LLMs for subsequent RAG.
2. Use the Spider or BirdBench dataset for SQL and use the CoDocBench dataset.
3. When evaluating natural language text, use the BLEU or BERT score for similarity. Similarly, when comparing synthesized programs against ground-truth, evaluate them using metrics like CodeBLEU or CodeBERTScore, but also execution accuracy of the generated program, where feasible.

References

1. codegen-350M-multi: <https://huggingface.co/Salesforce/codegen-350M-multi>
2. CoDocBench: <https://github.com/kunpai/codocbench>
3. Spider: <https://spider2-sql.github.io/>
4. BirdBench: <https://bird-bench.github.io/>
5. CodeParrot: <https://huggingface.co/datasets/codeparrot/github-code>
6. CodeBLEU: <https://arxiv.org/abs/2009.10297>

7. CodeBERTScore: <https://arxiv.org/abs/2302.05527>

8. Datasets: <https://github.com/codefuse-ai/Awesome-Code-LLM#8-datasets>

Check Points for the Project:

- **Check Point-1:** Documentation Generation + Training on Subset for a new language (2 weeks)
- **Check Point-2:** SQL generation + training on complete data on new language (5 week)
- **Check Point-3:** RAG System + Trained Model Evaluation (8 week)
- **Check Point-4:** Final Deployment and overall system evaluation (Put their model into the RAG system and evaluate) (11 week)

A Few Points to Note While Working On The Capstone Project:

- Though this is a group project, faculty/mentors can access individual performance/contributions and may award different marks to each individual of the same group.
- **Attendance is the minimum criterion for getting marks during the mentoring sessions.**
- **Project Proposal Format:** It consists of a document not more than 7/8-pages, covering the title, problem statement, literature review, methodologies, possible outcomes in stages, and applicability in the real world. The first page should contain the title and group number & team members' names.
- **Final Report/PPT Format:** It consists of a Word Document. & PPT, covering the title, problem statement, literature review, methodologies, outcomes in stages, final outcomes, challenges, and applicability in the real world. The first page should contain the title, group number & team members' names.
- Demonstration of the deployed model is a must, along with the final presentation and report.
- **There will be four checkpoints with the following expectations:**
 - **Check Point-1:** Preliminary Model & Training (on subset of data)
 - **Check Point-2:** Model building, training for the complete dataset
 - **Check Point-3:** Deployment testing (on a subset of data)
 - **Check Point-4:** Fine tuning & Final deployment